

Lab 9

In this lab, we're returning to the fork-join solution technique, but using it for operations other than sum.

Other Reductions using Fork-Join

Begin by downloading the given code (`lab9given.zip`). Unzip this, open the `lab9-reductions` folder it contains, and open the code as a project by double clicking on `lab9-reductions.code-workspace`. The only code this has initially is `SumArray.java`, the same program we worked with last week to sum the contents of an array. (I did remove the restriction that all the array values be between 0 and 9.)

Today, we'll look at changing this code to perform different operations. All the operations will have a similar structure; they are all called *reductions* because they reduce an array of input values to a single result.

For the first modification, we will find the minimum. (We discussed this in class yesterday.) You can either modify `SumArray` or make a copy of it with a more descriptive name. Either way, change the `compute` function so that it returns the minimum value rather than the sum. This requires small changes to both the base case (find the minimum of less than 100 values) and the recursive one (return the minimum of `left` and `right` rather than their sum). You will also want to change the serial computation in `main` so that you can check your answer.

Once you have that working, change the program again so that it returns both the minimum and the index where the minimum value occurs. To return multiple values, you'll need to create a class to store them both. Then use this class as the type of `RecursiveTask` your code extends (modify at the top of the `SumArray` class) and as the return value of `compute`. Then, you'll again modify the base case and recursion step of `compute` itself, as well as the serial implementation.

Next, we'll implement the reduction of finding the k^{th} smallest value. ($k = 1$ means finding the minimum, $k = 2$ means finding the second smallest, $k = 3$ means finding the 3rd smallest, etc.) Begin by implementing the serial version of this, possibly starting specifically with $k = 2$ (finding the second smallest value). To do this, you'll need to keep track of all k of the smallest values in order. Then, when you read a new value, compare it with these to see if the new value fits among these. To implement this function for general k , you'll need an array to keep track of the k smallest.

Once you've got the serial code for k^{th} smallest working, implement it in the fork-join framework. The object returned by `compute` will now need to contain the array with the k smallest. The base case will use your serial solution. To combine `left` and `right` in the recursion step, you'll want to do something like the `merge` routine in mergesort; the smallest value is one of the smallest values in the left and right solutions, the next smallest is either the other smallest value or the second smallest in the array you took the overall smallest from, etc.

If you have more time, I have a challenge problem that goes beyond reductions (but is related to them). The goal is to implement *prefix sums*: create a new array whose value at index i is the sum of all the values in the original array $\leq i$. For example, if the input is

3 1 4 2 5 3 1 3 4

then the array of prefix sums is

3 4 8 10 15 18 19 22 26

To implement this in parallel, you'll want to go back to the original `SumArray` except that you should shrink the array and recursion threshold so you can see what is happening. I suggest you also use `%` to restrict

the array values to a small range. Then modify the code to take a second array; the original array is the input and the new array will be the output. You'll need 2 passes. The first pass is essentially the existing **SumArray** except that it should write its sum to the last cell of its part of the array in addition to returning it.

For the second pass, you'll need a different extension of the **RecursiveTask** class with the same recursion structure as your modified **SumArray**. Tasks of this new class will take the sum of all the values before its part of the array as an additional argument. With this extra information, its base case will be able to fill in the second array. For the recursion step, you'll need to read the array values written in the first pass to calculate all the appropriate arguments.